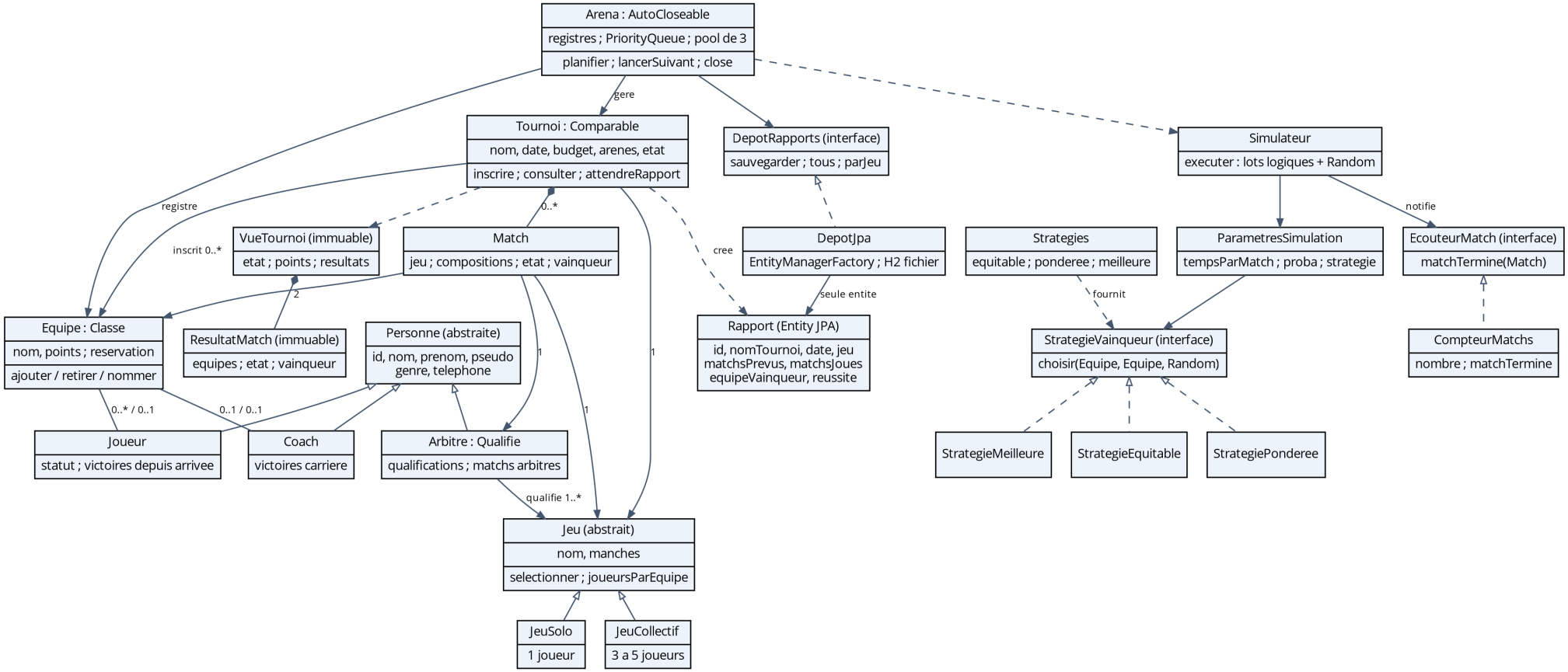
IMT Arena — Dossier de conception

1. Modèle objet et relations



Lecture UML. Triangle vide : héritage (trait pointillé : réalisation) ; losange plein : composition. Multiplicités portées sur les associations. Classe et Qualifie sont des interfaces de capacités ; Jeu et Personne sont abstraites. Le diagramme Mermaid complet et éditable est fourni dans classes.mmd.
Invariants. Une personne possède un id unique. Joueur et Coach ne peuvent rejoindre deux équipes ; les mutations passent par Equipe. La réservation fige les membres pendant le tournoi. Match valide adversaires distincts, effectifs et qualification ; une victoire donne trois points, un incident aucun. Après trois victoires depuis son arrivée, un amateur devient professionnel ; les victoires du coach restent acquises après transfert.

2. Collections, patterns et concurrence

Besoin / structure	Justification et précaution
Tournois : PriorityQueue<Tournoi>	Comparable sur date puis nom immuables ; tête O(1), ajout/retrait O(log n). Liste affichée triée O(n log n), car itération de la file non triée.
Registres : HashMap (personnes : LinkedHashMap)	Recherche moyenne O(1), rejet des doublons par nom ou id. LinkedHashMap préserve l'ordre des arbitres : tirage reproductible.
Qualifications : Set<Jeu> immuable	Unicité, appartenance moyenne O(1). Jeux extensibles : EnumSet inadapté. Aucune modification externe.
Effectifs / matchs : ArrayList	Ordre stable, accès O(1), ajout amorti O(1). Copies List.copyOf ; aucune liste interne exposée.
Classement : List triée + Comparator	Points décroissants puis nom ; O(n log n). Points capturés avant tri. Pas de TreeSet indexé sur des points mutables.
Gagnants : LinkedHashMap<Equipe,List<Match>>	Regroupement dans l'ordre du classement, listes immuables. Incidents exclus. Coût O(e*m), suffisant pour quelques équipes.

Patterns utiles. Strategy sépare le vainqueur du déroulement : hasard équitable, pondération par points ou meilleure équipe. Trois petites classes implémentent une interface avec des conditions simples. Observer diffuse résultat/incident vers des composants indépendants (exemple : CompteurMatchs) ; callbacks hors verrou, exceptions collectées. Ils doivent être rapides et coopératifs. Factory/Builder non retenus : constructeurs courts et validants suffisants ; pas de couche ajoutée sans besoin.

Ordonnancement et sûreté. Un ExecutorService fixe exécute au plus trois tournois. Le moniteur court d'Arena atomise priorité, capacité et lancement ; chaque équipe est réservée sous son propre moniteur, avec rollback en cas d'échec. Aucun verrou d'équipe ne couvre sommeil, callback ou JPA. Appartenances exclusives + effectifs figés empêchent la double mobilisation des joueurs/coachs. Les mises à jour de résultat et points locaux sont sérialisées par Tournoi ; consulter() retourne une photographie immuable. L'attente du CompletableFuture est bornée et couvre la sauvegarde. close() ferme les soumissions, attend, puis interrompt au besoin ; le dépôt ferme après les workers.

Simulation et ambiguïtés. Random privé et graine par tournoi ; rencontres et arbitres d'ordre stable. Arènes logiques par lots : durée M*t/A, dernier lot proratisé (15*100/3 = 500 ms), hors surcoûts CPU/JPA. Le sujet n'impose pas un planning exclusif des arbitres/équipes par match. Disponible signifie non engagé dans un tournoi ; arbitrés compte les matchs joués. Réussite : 10*joués >= 7*prévus. Vainqueur : points locaux puis nom ; aucun si zéro match joué.

Persistance et validation. Rapport est la seule entité, sans association vers le modèle vivant. DepotRapports découple JPA ; DepotJpa partage la factory et utilise un EntityManager par opération, transaction avec rollback. H2 est un fichier. Les tests JUnit couvrent règles, erreurs, hasard, concurrence, fermeture et réouverture du fichier ; le détail des résultats réellement exécutés figure dans VALIDATION.md.